MODUL 00 · INDEKS DAN PETA BELAJAR

Perancangan dan Analisis Algoritma: Kumpulan Modul Pembelajaran

24 modul belajar mandiri yang dikembangkan dari 16 bahan kuliah, dari konsep dasar algoritma sampai algoritma lintasan terpendek.

ISI KUMPULAN MODUL

- 24 modul PDF, total 248 halaman.
- 178 soal kuis beserta kunci jawaban dan pembahasan.
- Setiap contoh kode Python di dalam modul dijalankan otomatis saat pembuatan PDF, dan keluarannya dicocokkan dengan yang tercetak.
- Setiap kesalahan yang ditemukan pada bahan sumber ditandai dalam kotak **Catatan Koreksi terhadap Sumber** (ringkasannya di bagian 5).

STRUKTUR SETIAP MODUL

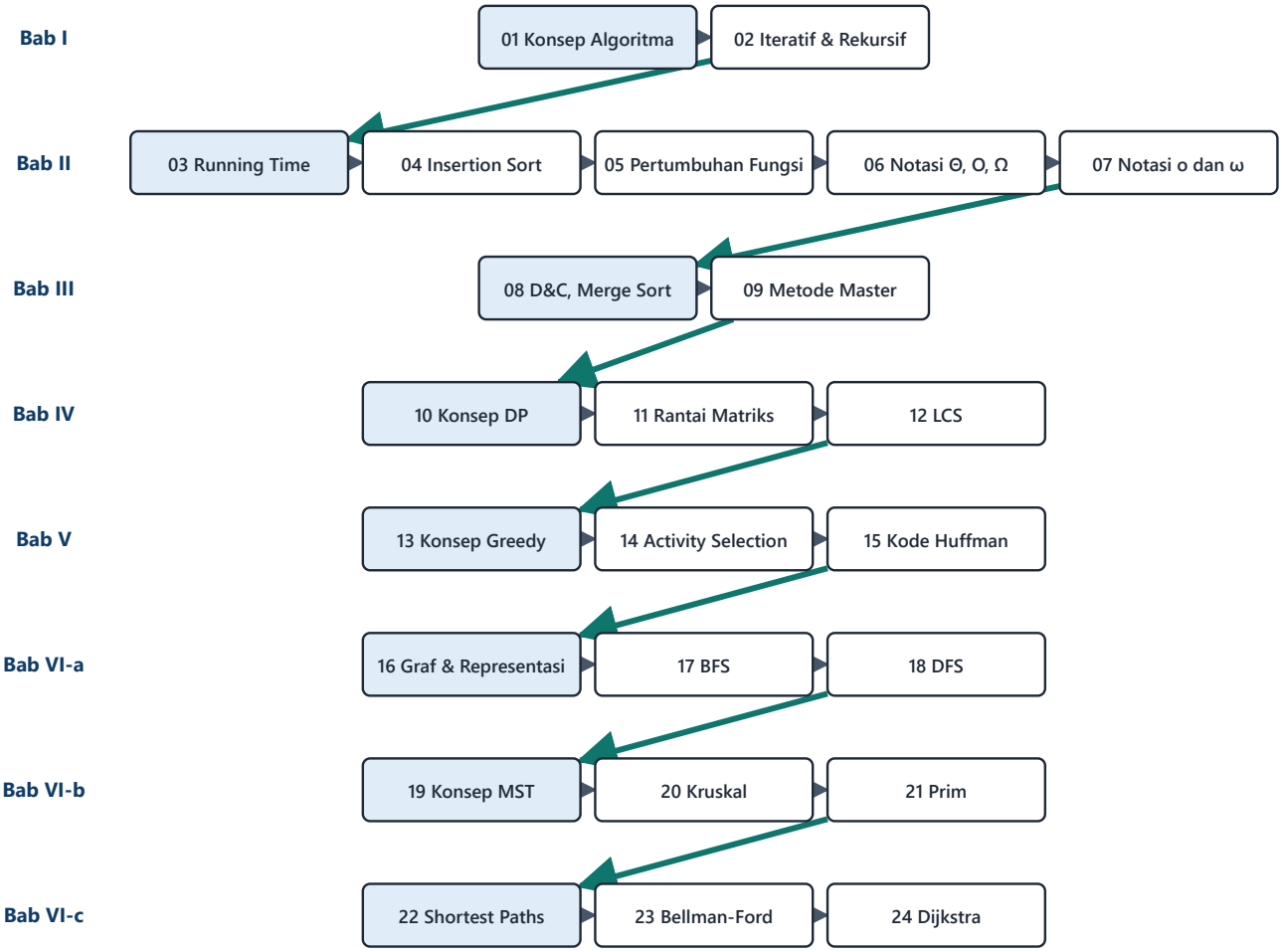
1. Identitas: judul, tujuan, prasyarat, materi asal
2. Pendahuluan
3. Pembahasan inti
4. Contoh dan penerapan
5. Analisis dan hubungan konsep
6. Kesimpulan
7. Kuis (halaman baru), lalu kunci jawaban (halaman baru)
8. Referensi

1. Cara Memakai Modul Ini

- **Ikuti urutan nomor modul.** Urutan disusun agar setiap konsep prasyarat dibahas lebih dulu. Kolom "Prasyarat" pada katalog (bagian 3) menyebutkan modul yang perlu dikuasai sebelumnya.
- **Modul konsep umum dan modul khusus dipisahkan.** Misalnya Modul 10 membahas konsep Dynamic Programming, sedangkan Modul 11 dan 12 menerapkannya pada masalah tertentu. Modul konsep umum ditandai kotak biru muda pada peta belajar.
- **Telusuri contoh dengan tangan.** Setiap algoritma disertai penelusuran langkah demi langkah. Cobalah mengerjakannya sendiri sebelum membaca tabel penelusuran.
- **Kerjakan kuis tanpa melihat kunci.** Kunci jawaban selalu berada di halaman setelah soal, lengkap dengan pembahasan.
- **Baca kotak koreksi.** Jika kamu juga memakai bahan kuliah asli, kotak merah muda menjelaskan bagian sumber yang perlu dibaca dengan hati-hati.

2. Peta Belajar

Panah hitam menunjukkan urutan di dalam satu bab; panah hijau menunjukkan perpindahan ke bab berikutnya. Kotak biru muda adalah modul konsep umum yang menjadi pintu masuk setiap bab.



Gambar 1. Urutan belajar 24 modul.

Bab	Fokus	Modul
Bab I · Dasar Algoritma	Fondasi: apa itu algoritma, cara menuliskannya, iterasi dan rekursi.	01, 02
Bab II · Analisis Running Time	Mengukur efisiensi: running time, pertumbuhan fungsi, dan notasi asimtotik.	03, 04, 05, 06, 07
Bab III · Divide and Conquer dan Recurrence	Paradigma pertama dan alat analisis algoritma rekursif.	08, 09
Bab IV · Dynamic Programming	Optimasi dengan menyimpan solusi submasalah yang tumpang tindih.	10, 11, 12
Bab V · Algoritma Greedy	Optimasi dengan pilihan terbaik lokal yang terbukti optimal.	13, 14, 15
Bab VI · Algoritma Graf	Representasi graf, penelusuran, MST, dan lintasan terpendek.	16, 17, 18, 19, 20, 21, 22, 23, 24

3. Katalog Modul

No	Judul dan deskripsi singkat	Prasyarat	Ukuran
Bab I · Dasar Algoritma — Fondasi: apa itu algoritma, cara menuliskannya, iterasi dan rekursi.			
01	**Konsep Dasar Algoritma: Definisi, Karakteristik, Representasi, dan Struktur** Fondasi untuk seluruh modul: apa itu algoritma, syarat algoritma yang baik, cara menuliskannya, dan tiga struktur pembangunnya.	Tidak ada prasyarat khusus. Modul ini adalah titik awal.	14 hlm 8 soal
02	**Algoritma Iteratif dan Rekursif: Studi Kasus Binary Search** Dua cara mengulang pekerjaan: dengan perulangan atau dengan fungsi yang memanggil dirinya sendiri.	Modul 01: pseudocode, struktur percabangan dan perulangan, penelusuran algoritma.	12 hlm 7 soal
Bab II · Analisis Running Time — Mengukur efisiensi: running time, pertumbuhan fungsi, dan notasi asimtotik.			
03	**Konsep Running Time dan Model Analisis Algoritma** Mengukur kecepatan algoritma tanpa stopwatch: menghitung biaya dan banyaknya eksekusi setiap langkah.	Modul 01: pseudocode dan struktur perulangan. Modul 02 (disarankan): penelusuran algoritma.	11 hlm 7 soal
04	**Insertion Sort: Cara Kerja dan Analisis Running Time** Algoritma pengurutan pertama kita, lengkap dengan penurunan running time kasus terbaik (linier) dan terburuk (kuadrat).	Modul 03: metode cost × jumlah operasi, kasus terbaik dan terburuk, deret aritmetika.	11 hlm 8 soal
05	**Pertumbuhan Fungsi Running Time** Seberapa cepat waktu eksekusi membengkak ketika data bertambah, dan mengapa memilih algoritma lebih penting daripada membeli prosesor lebih cepat.	Modul 03: running time $T(n)$ dan orde pertumbuhan. Modul 04: contoh running time linier dan kuadrat.	9 hlm 7 soal
06	**Notasi Asimtotik: Theta (Θ), Big-Oh (O), dan Omega (Ω)** Bahasa formal untuk menyatakan laju pertumbuhan running time: batas ketat, batas atas, dan batas bawah.	Modul 05: bentuk-bentuk fungsi running time dan rasio pertumbuhan. Modul 04: hasil running time Insertion Sort ($an + b$ dan $an^2 + bn + c$).	11 hlm 8 soal
07	**Notasi Little-o (o) dan Little-Omega (ω)** Membedakan batas yang ketat dan yang tidak ketat, serta memakai limit untuk membandingkan laju pertumbuhan fungsi.	Modul 06: definisi Θ, O, Ω. Konsep limit dasar ($\lim_{n\to\infty} 1/n = 0$).	9 hlm 7 soal
Bab III · Divide and Conquer dan Recurrence — Paradigma pertama dan alat analisis algoritma rekursif.			
08	**Paradigma Divide and Conquer dan Merge Sort** Memecah masalah menjadi submasalah, menyelesaikannya secara rekursif, lalu menggabungkan hasilnya.	Modul 02: rekursi dan call stack. Modul 04: Insertion Sort (sebagai pembanding). Modul 06: notasi Θ.	12 hlm 8 soal

09	**Recurrence dan Metode Master** Menentukan notasi asimtotik running time algoritma rekursif berbentuk $T(n) = aT(n/b) + f(n)$ secara cepat dan sistematis.	Modul 06–07: notasi Θ, O, Ω, dan perbandingan secara polinomial. Modul 08: recurrence Merge Sort dan pohon rekursi.	10 hlm 8 soal
Bab IV · Dynamic Programming — Optimasi dengan menyimpan solusi submasalah yang tumpang tindih.			
10	**Konsep Dynamic Programming** Menyelesaikan masalah optimasi dengan menyimpan solusi submasalah agar tidak dihitung berulang kali.	Modul 02: rekursi. Modul 08: paradigma Divide and Conquer. Modul 05–06: notasi asimtotik dan pertumbuhan eksponensial.	9 hlm 7 soal
11	**Perkalian Rantai Matriks (Matrix-Chain Multiplication)** Menentukan urutan perkalian matriks yang meminimalkan banyaknya operasi perkalian skalar dengan Dynamic Programming.	Modul 10: konsep dan empat langkah Dynamic Programming. Dasar aljabar matriks (perkalian matriks).	11 hlm 8 soal
12	**Longest Common Subsequence (LCS)** Mencari subbarisan persekutuan terpanjang dari dua barisan dengan tabel Dynamic Programming dan menelusurinya kembali dengan PRINT-LCS.	Modul 10: konsep dan empat langkah Dynamic Programming. Modul 02: rekursi (untuk PRINT-LCS).	11 hlm 8 soal
Bab V · Algoritma Greedy — Optimasi dengan pilihan terbaik lokal yang terbukti optimal.			
13	**Konsep Algoritma Greedy** Memilih yang terlihat terbaik saat ini: kapan strategi sederhana ini menghasilkan solusi optimal, dan kapan gagal.	Modul 10: konsep Dynamic Programming dan masalah optimasi.	8 hlm 7 soal
14	**Activity-Selection Problem dengan Algoritma Greedy** Menjadwalkan sebanyak mungkin aktivitas pada satu sumber daya dengan memilih aktivitas yang paling cepat selesai.	Modul 13: konsep Greedy dan greedy-choice property. Modul 04/08: pengurutan dalam waktu $O(n \lg n)$.	10 hlm 7 soal
15	**Kode Huffman: Kompresi Data dengan Algoritma Greedy** Memberi kode pendek untuk karakter yang sering muncul dan kode panjang untuk yang jarang, tanpa kehilangan kemampuan decode.	Modul 13: konsep Greedy. Pengetahuan dasar bilangan biner dan pohon biner.	10 hlm 8 soal
Bab VI · Algoritma Graf — Representasi graf, penelusuran, MST, dan lintasan terpendek.			
16	**Graf dan Representasinya: Adjacency List dan Adjacency Matrix** Istilah dasar graf dan dua cara menyimpannya di komputer, sebagai fondasi semua algoritma graf.	Modul 03 dan 06: running time dan notasi asimtotik.	9 hlm 7 soal
17	**Breadth-First Search (BFS)** Menelusuri graf lapis demi lapis untuk menemukan lintasan dengan jumlah busur paling sedikit dari satu simpul awal.	Modul 16: graf dan adjacency-list.	11 hlm 8 soal
18	**Depth-First Search (DFS)** Menelusuri graf sedalam mungkin lalu mundur (backtrack), mencatat waktu ditemukan dan waktu selesai setiap simpul.	Modul 16: graf dan adjacency-list. Modul 17: BFS (sebagai pembanding). Modul 02: rekursi dan call stack.	11 hlm 8 soal

19	**Konsep Minimum Spanning Tree (MST)** Menghubungkan semua simpul dengan total bobot sekecil mungkin: definisi, sifat, dan dasar teoretis algoritma Kruskal dan Prim.	Modul 16: graf, graf berbobot, dan tree. Modul 13: konsep Greedy.	9 hlm 7 soal
20	**Algoritma Kruskal untuk Minimum Spanning Tree** Memilih busur dari yang paling ringan, menolak yang membentuk cycle, sampai semua simpul tergabung dalam satu tree.	Modul 19: konsep MST, cut, light edge, safe edge. Modul 08: pengurutan $O(n \lg n)$.	10 hlm 7 soal
21	**Algoritma Prim untuk Minimum Spanning Tree** Menumbuhkan satu tree dari sebuah akar dengan selalu mengambil simpul terdekat yang belum masuk tree.	Modul 19: konsep MST, cut, dan light edge. Modul 16: adjacency-list graf berbobot. Modul 20 (disarankan): algoritma Kruskal sebagai pembanding.	10 hlm 7 soal
22	**Single-Source Shortest Paths: Konsep dan Relaksasi** Apa arti lintasan terpendek pada graf berbobot, peran bobot dan cycle negatif, serta operasi RELAX yang menjadi inti Bellman-Ford dan Dijkstra.	Modul 16: graf berarah berbobot. Modul 17: BFS (jarak dalam jumlah busur, predecessor).	9 hlm 7 soal
23	**Algoritma Bellman-Ford** Lintasan terpendek dari satu sumber yang mengizinkan bobot negatif dan mampu mendeteksi cycle berbobot negatif.	Modul 22: jarak terpendek, cycle negatif, INITIALIZE-SINGLE-SOURCE, dan RELAX.	10 hlm 7 soal
24	**Algoritma Dijkstra** Lintasan terpendek dari satu sumber untuk bobot nonnegatif: selalu selesaikan simpul yang jaraknya paling kecil lebih dulu.	Modul 22: jarak terpendek, INITIALIZE-SINGLE-SOURCE, RELAX. Modul 23 (disarankan): Bellman-Ford sebagai pembanding. Modul 13: konsep Greedy.	11 hlm 7 soal

4. Pemetaan Bahan Sumber ke Modul

Pemecahan modul mengikuti konsep inti, bukan jumlah file sumber. Satu file sumber dapat menjadi beberapa modul (misalnya Materi 12 menjadi konsep MST, Kruskal, dan Prim), dan satu modul dapat memakai beberapa sumber (Modul 22 menggabungkan bagian pengantar Materi 13.1 dan 13.2).

Bahan sumber	Modul
PENGERTIAN ALGORITMA PEMROGRAMAN	01, 02 (dan bagian kompleksitas di 03, 05)
Materi 2. ALGORITMA INSERTION SORT dan RUNNING TIME	03, 04
Materi 3. PERTUMBUHAN FUNGSI RUNNING TIME	05
Materi 4. NOTASI ASIMTOTIK	06
Materi 5 Notasi Asimtotik Little-Oh dan Little-Omega	07
Materi 6. Algoritma Divide and Conquer	08
Materi 7. Recurrence Metode Master	09

Materi 8 DYNAMIC PROGRAMMING Perkalian Rantai Matriks	10, 11
Materi 9 DYNAMIC PROGRAMMING Longest Common Subsequence	12
Materi 10.1 ALGORITMA GREEDY An Activity Selection Problem	13, 14
Materi 10.2 ALGORITMA GREEDY-ALGORITMA HUFFMAN	15
Materi 11.1 ALGORITMA GRAF, BFS	16, 17
Materi 11.2 ALGORITMA GRAF, DFS	18
Materi 12. MINIMUM SPANNING TREE, algoritma Kruskal dan Prim	19, 20, 21
Materi 13.1 SINGLE SOURCE SHORTEST PATHS, Algoritma Bellman Ford	22, 23
Materi 13.2 SINGLE SOURCE SHORTEST PATHS, Algoritma Dijkstra	22, 24

TAMBAHAN DI LUAR BAHAN SUMBER

Agar setiap modul dapat dipelajari mandiri, beberapa penjelasan ditambahkan dari pustaka yang dapat diverifikasi (terutama Cormen dkk., 2009, yang juga menjadi rujukan bahan kuliah), misalnya: invarian perulangan (Modul 04), aturan L'Hôpital (07), pohon rekursi dan batasan Metode Master (09), contoh pemotongan batang (10), contoh penyangkal Greedy (13), klasifikasi busur DFS (18), teorema cut dan safe edge (19), himpunan saling lepas (20), sifat-sifat relaksasi (22), serta contoh kegagalan Dijkstra pada bobot negatif (24). Bagian tambahan disebutkan sebagai tambahan di dalam modul.

5. Ringkasan Koreksi terhadap Bahan Sumber

Bagian sumber berikut mengandung salah ketik atau penjelasan yang kurang tepat. Makna sumber tidak diubah diam-diam: setiap koreksi dijelaskan di modul terkait.

Modul	Koreksi
05	Soal studi kasus menyebut prosesor "2 kali lebih cepat", tetapi penyelesaiannya memakai "20 kali" (65536T/20). Kedua versi dihitung; kesimpulan tetap: memilih algoritma lebih efisien.
06	Kalimat penutup bukti $\frac{1}{2}n^2 - 3n = O(n^2)$ tertulis "$0 \leq 2n^2 - 3n + 1 \leq cn^2$" (terbawa dari contoh Theta).
08	Tertulis rekursi berhenti "apabila nilai $p < r$"; yang benar berhenti ketika $p \geq r$. Latihan penelusuran {3,10,8,20,5} yang belum selesai dilengkapi.
09	Rentang ε pada contoh kasus 1 ditulis $0 < \varepsilon < \frac{1}{2}$; nilai $\varepsilon = \frac{1}{2}$ juga memenuhi (catatan kecil).
11	Penjelasan bahwa komputer menghitung 5×3 dengan penjumlahan berulang tidak tepat untuk prosesor modern; diberikan alasan yang benar mengapa perkalian skalar dihitung.
12	Rumus langkah 11 tertulis $c[i, j] = c[i-1, j] + c[i, j-1] + 1$; yang benar $c[i, j] = c[i-1, j-1] + 1$. Pemanggilan awal PRINT-LCS tertulis (b,X,8,9); yang benar (b,X,7,6).
14	Syarat aktivitas berikutnya tertulis "lebih besar dari" waktu selesai; yang benar "lebih besar atau sama dengan" ($s[m] \geq f[k]$). Salah ketik $f[4] = 4$ (seharusnya 7) dan "Loop 10: m = 12".
15	Keterangan loop 4 ("simpul 55 disisipkan antara a dan simpul 55") diluruskan: simpul 55 ditempatkan setelah a:45.

20	Hasil MAKE-SET tertulis "8 tree", padahal daftar dan jumlah simpulnya 9.

6. Daftar Pustaka Gabungan

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Knuth, D. E. (1997). *The Art of Computer Programming, Vol. 1: Fundamental Algorithms* (3rd ed.). Reading, MA: Addison-Wesley. (Bagian 1.1 tentang lima sifat algoritma.)
3. Kleinberg, J., & Tardos, É. (2006). *Algorithm Design*. Boston: Pearson/Addison-Wesley.
4. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Upper Saddle River, NJ: Addison-Wesley.
5. Bellman, R. (1957). *Dynamic Programming*. Princeton, NJ: Princeton University Press.
6. Knuth, D. E. (1976). Big Omicron and Big Omega and Big Theta. *ACM SIGACT News*, 8(2), 18–24. https://doi.org/10.1145/1008328.1008329
7. Turing, A. M. (1937). On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1), 230–265. https://doi.org/10.1112/plms/s2-42.1.230
8. Bentley, J. L., Haken, D., & Saxe, J. B. (1980). A General Method for Solving Divide-and-Conquer Recurrences. *ACM SIGACT News*, 12(3), 36–44. https://doi.org/10.1145/1008861.1008865
9. Huffman, D. A. (1952). A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE*, 40(9), 1098–1101. https://doi.org/10.1109/JRPROC.1952.273898
10. Moore, E. F. (1959). The Shortest Path Through a Maze. Dalam *Proceedings of the International Symposium on the Theory of Switching* (hlm. 285–292). Cambridge, MA: Harvard University Press.
11. Tarjan, R. E. (1972). Depth-First Search and Linear Graph Algorithms. *SIAM Journal on Computing*, 1(2), 146–160. https://doi.org/10.1137/0201010
12. Kruskal, J. B. (1956). On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem. *Proceedings of the American Mathematical Society*, 7(1), 48–50. https://doi.org/10.1090/S0002-9939-1956-0078686-7
13. Prim, R. C. (1957). Shortest Connection Networks and Some Generalizations. *Bell System Technical Journal*, 36(6), 1389–1401. https://doi.org/10.1002/j.1538-7305.1957.tb01515.x
14. Bellman, R. (1958). On a Routing Problem. *Quarterly of Applied Mathematics*, 16(1), 87–90. https://doi.org/10.1090/qam/102435
15. Ford, L. R., Jr. (1956). *Network Flow Theory* (Paper P-923). Santa Monica, CA: RAND Corporation.
16. Dijkstra, E. W. (1959). A Note on Two Problems in Connexion with Graphs. *Numerische Mathematik*, 1, 269–271. https://doi.org/10.1007/BF01386390
17. Fredman, M. L., & Tarjan, R. E. (1987). Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms. *Journal of the ACM*, 34(3), 596–615. https://doi.org/10.1145/28869.28874
18. Hu, T. C., & Shing, M. T. (1982). Computation of Matrix Chain Products. Part I. *SIAM Journal on Computing*, 11(2), 362–373. https://doi.org/10.1137/0211028